INSTITUTO FEDERAL
Brasília

BOLSA FUTURO DIGITAL

GUIA PRÁTICO

Criação de uma API REST em Python utilizando FastAPI

Curso Bolsa Futuro Digital

Material didático para atividade prática

1. Apresentação

Este guia foi elaborado para apoiar os estudantes do curso Bolsa Futuro Digital na criação de uma API REST utilizando Python e FastAPI. A proposta é construir, passo a passo, uma API de gerenciamento de livros, relacionando a prática aos conceitos de HTTP, requisição e resposta, métodos HTTP, JSON, endpoints e operações CRUD.

Objetivo didático: Ao final da prática, o estudante deverá conseguir executar um servidor FastAPI, criar endpoints, receber e retornar dados em JSON, utilizar GET, POST, PUT e DELETE e testar a API pela documentação automática.

2. O que vamos construir

A aplicação criada neste guia será uma API de livros. Ela disponibilizará operações para listar, consultar, cadastrar, atualizar e excluir livros.

Método HTTP	Endpoint	Finalidade
GET	/livros	Listar todos os livros
GET	/livros/{id}	Consultar um livro
POST	/livros	Cadastrar um novo livro
PUT	/livros/{id}	Atualizar um livro
DELETE	/livros/{id}	Excluir um livro

3. Conceitos essenciais

3.1 Cliente, servidor, requisição e resposta

Uma API funciona, de forma simplificada, como uma interface de comunicação entre sistemas. O cliente envia uma requisição HTTP para um servidor. O servidor processa a solicitação e devolve uma resposta.

```
Cliente
   |
   | Requisição HTTP
```

```
   v
API FastAPI
   |
   | Processamento
   v
Resposta HTTP + JSON
   |
   v
Cliente
```

3.2 Métodos HTTP e CRUD

Método	CRUD	Uso comum
GET	Read	Consultar dados
POST	Create	Criar dados
PUT	Update	Atualizar um recurso
PATCH	Update	Atualizar parcialmente
DELETE	Delete	Excluir um recurso

3.3 JSON

JSON é um formato textual muito utilizado em APIs REST para representar e trocar dados. Ele organiza as informações principalmente em pares de chave e valor.

```
{
    "id": 1,
    "titulo": "Dom Casmurro",
    "autor": "Machado de Assis"
}
```

4. Pré-requisitos

- Python instalado no computador.
- Acesso ao terminal, Prompt de Comando, PowerShell ou terminal Linux.
- Editor de código. Recomenda-se Visual Studio Code ou equivalente.
- Conhecimentos básicos de Python, funções, listas e dicionários.

5. Preparação do ambiente

5.1. Verificar a instalação do Python

Abra o terminal e execute:

```
python --version
```

Em algumas distribuições Linux, use:

```
python3 --version
```

5.2 Criar a pasta do projeto

```
mkdir minha_api
cd minha_api
```

5.3 Criar um ambiente virtual

No Windows:

```
python -m venv venv
venv\Scripts\activate
```

No Linux:

```
python3 -m venv venv
source venv/bin/activate
```

Por que usar ambiente virtual?: O ambiente virtual isola as bibliotecas do projeto e evita conflitos com outros projetos Python instalados no mesmo computador.

5.4 Instalar FastAPI e Uvicorn

```
pip install fastapi uvicorn
```

O FastAPI será o framework da API. O Uvicorn será o servidor responsável por executar a aplicação.

6. Criando a primeira API

Crie um arquivo chamado main.py dentro da pasta do projeto e insira o código abaixo:

```
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
def inicio():
    return {"mensagem": "Minha primeira API com FastAPI"}
```

6.1 Entendendo o código

- from fastapi import FastAPI: importa a classe FastAPI.
- app = FastAPI(): cria a aplicação.
- @app.get("/"): define uma rota que responde ao método HTTP GET.
- def inicio(): define a função executada quando a rota é acessada.
- return {...}: devolve um dicionário Python, convertido automaticamente para JSON.

7. Executando a API

No terminal, na mesma pasta do arquivo main.py, execute:

```
uvicorn main:app --reload
```

A aplicação deverá ficar disponível em:

```
http://127.0.0.1:8000
```

Sobre --reload: Essa opção reinicia automaticamente o servidor quando o código é alterado. É adequada para desenvolvimento e atividades de laboratório.

8. Documentação automática

O FastAPI gera automaticamente uma interface de documentação e testes. Com o servidor em execução, acesse:

```
http://127.0.0.1:8000/docs
```

Também é possível acessar uma documentação alternativa em:

```
http://127.0.0.1:8000/redoc
```

Na página /docs, cada endpoint pode ser aberto, testado pelo botão “Try it out” e executado pelo botão “Execute”.

9. Construindo a API de livros

9.1 Criar dados iniciais

Substitua o conteúdo de main.py pelo código abaixo:

```
from fastapi import FastAPI

app = FastAPI()

livros = [
    {"id": 1, "titulo": "Dom Casmurro", "autor": "Machado de Assis"},
    {"id": 2, "titulo": "O Cortiço", "autor": "Aluísio Azevedo"},
    {"id": 3, "titulo": "Vidas Secas", "autor": "Graciliano Ramos"}
]


@app.get("/")
def inicio():
    return {"mensagem": "API de Livros"}


@app.get("/livros")
```

```
def listar_livros():
    return livros
```

Abra no navegador:

```
http://127.0.0.1:8000/livros
```

A resposta será uma lista JSON contendo os livros cadastrados em memória.

10. Consultando um livro específico

Adicione o endpoint abaixo ao final do arquivo:

```
@app.get("/livros/{livro_id}")
def buscar_livro(livro_id: int):
    for livro in livros:
        if livro["id"] == livro_id:
            return livro

    return {"mensagem": "Livro não encontrado"}
```

Teste, por exemplo:

```
http://127.0.0.1:8000/livros/2
```

O valor 2 é recebido pela variável livro_id. O FastAPI também verifica que o valor deve ser inteiro por causa da anotação livro_id: int.

11. Criando um modelo de dados com Pydantic

Para receber dados enviados pelo cliente, utilizaremos BaseModel, fornecido pelo Pydantic e integrado ao FastAPI.

```
from pydantic import BaseModel


class Livro(BaseModel):
```

```
    titulo: str
    autor: str
```

Esse modelo estabelece que cada novo livro deve conter titulo e autor, ambos do tipo texto.

12. Cadastrando livros com POST

Adicione o endpoint:

```
@app.post("/livros", status_code=201)
def cadastrar_livro(livro: Livro):
    novo_livro = {
        "id": len(livros) + 1,
        "titulo": livro.titulo,
        "autor": livro.autor
    }

    livros.append(novo_livro)
    return novo_livro
```

Para testar, abra /docs, selecione POST /livros, clique em “Try it out” e envie:

```
{
    "titulo": "Memórias Póstumas de Brás Cubas",
    "autor": "Machado de Assis"
}
```

O status 201 indica que um novo recurso foi criado com sucesso.

13. Atualizando um livro com PUT

```
@app.put("/livros/{livro_id}")
def atualizar_livro(livro_id: int, livro_atualizado: Livro):
    for livro in livros:
        if livro["id"] == livro_id:
            livro["titulo"] = livro_atualizado.titulo
            livro["autor"] = livro_atualizado.autor
            return livro
```

```
    return {"mensagem": "Livro não encontrado"}
```

Teste pelo /docs informando um id e enviando, por exemplo:

```
{
    "titulo": "Dom Casmurro - Edição Especial",
    "autor": "Machado de Assis"
}
```

14. Excluindo um livro com DELETE

```
@app.delete("/livros/{livro_id}")
def excluir_livro(livro_id: int):
    for livro in livros:
        if livro["id"] == livro_id:
            livros.remove(livro)
            return {"mensagem": "Livro excluído com sucesso"}

    return {"mensagem": "Livro não encontrado"}
```

Teste pelo /docs usando um id existente. Depois execute GET /livros para conferir se o recurso foi removido.

15. Tratando erros corretamente

Em uma API, é melhor retornar códigos HTTP adequados. Quando o recurso não existir, podemos utilizar HTTPException.

```
from fastapi import HTTPException


@app.get("/livros/{livro_id}")
def buscar_livro(livro_id: int):
    for livro in livros:
        if livro["id"] == livro_id:
            return livro
```

```
    raise HTTPException(
        status_code=404,
        detail="Livro não encontrado"
    )
```

Assim, uma consulta como GET /livros/999 poderá retornar o código 404 Not Found.

16. Código completo da API

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="API de Livros")


class Livro(BaseModel):
    titulo: str
    autor: str


livros = [
    {"id": 1, "titulo": "Dom Casmurro", "autor": "Machado de Assis"},
    {"id": 2, "titulo": "O Cortiço", "autor": "Aluísio Azevedo"},
    {"id": 3, "titulo": "Vidas Secas", "autor": "Graciliano Ramos"}
]


@app.get("/")
def inicio():
    return {"mensagem": "API de Livros"}


@app.get("/livros")
def listar_livros():
    return livros


@app.get("/livros/{livro_id}")
def buscar_livro(livro_id: int):
    for livro in livros:
```

```
        if livro["id"] == livro_id:
            return livro

    raise HTTPException(status_code=404, detail="Livro não encontrado")


@app.post("/livros", status_code=201)
def cadastrar_livro(livro: Livro):
    novo_livro = {
        "id": len(livros) + 1,
        "titulo": livro.titulo,
        "autor": livro.autor
    }
    livros.append(novo_livro)
    return novo_livro


@app.put("/livros/{livro_id}")
def atualizar_livro(livro_id: int, livro_atualizado: Livro):
    for livro in livros:
        if livro["id"] == livro_id:
            livro["titulo"] = livro_atualizado.titulo
            livro["autor"] = livro_atualizado.autor
            return livro

    raise HTTPException(status_code=404, detail="Livro não encontrado")


@app.delete("/livros/{livro_id}")
def excluir_livro(livro_id: int):
    for livro in livros:
        if livro["id"] == livro_id:
            livros.remove(livro)
            return {"mensagem": "Livro excluído com sucesso"}

    raise HTTPException(status_code=404, detail="Livro não encontrado")
```

17. Sequência recomendada de testes

1. Execute GET /livros e observe a lista inicial.

2. Execute GET /livros/1 e consulte um recurso específico.
3. Execute POST /livros e cadastre um novo livro.
4. Execute novamente GET /livros e confirme o cadastro.
5. Execute PUT /livros/1 e altere os dados do livro.
6. Execute GET /livros/1 e confirme a atualização.
7. Execute DELETE /livros/2.
8. Execute GET /livros e confirme a exclusão.
9. Execute GET /livros/999 e observe o erro 404.

18. O que aprendemos

Conceito	Aplicação na prática
HTTP	Protocolo utilizado nas requisições e respostas.
Endpoint	Endereço disponibilizado pela API, como /livros.
GET	Consulta informações.
POST	Cria um novo recurso.
PUT	Atualiza um recurso existente.
DELETE	Remove um recurso.
JSON	Formato utilizado para representar e trocar dados.
Status HTTP	Indica o resultado da operação, como 200, 201 ou 404.
Swagger /docs	Interface automática para documentar e testar endpoints.

19. Limitação do exemplo: dados em memória

Neste guia, os livros estão armazenados em uma lista Python. Isso significa que os dados existem apenas enquanto o programa está em execução. Se o servidor for encerrado e iniciado novamente, os cadastros feitos durante a execução serão perdidos.

Importante: Em uma aplicação real, normalmente utilizamos um banco de dados, como SQLite,

PostgreSQL ou MySQL, para persistir os dados.

20. Problemas comuns e soluções

Problema	Solução sugerida
“python” não é reconhecido	Verifique se o Python está instalado e adicionado ao PATH. No Linux, tente python3.
“uvicorn” não é reconhecido	Confirme se o ambiente virtual está ativo e execute pip install fastapi uvicorn.
Erro ao importar FastAPI	Ative o ambiente virtual e confirme a instalação com pip show fastapi.
A página não abre	Confirme se o servidor continua em execução e se o terminal informa http://127.0.0.1:8000.
Alterei o código, mas nada mudou	Use uvicorn main:app --reload e salve o arquivo.
Erro 404	Verifique se a rota digitada existe e se o parâmetro informado está correto.

21. Atividade de fixação

Crie uma nova API REST inspirada na API de livros. Escolha um domínio diferente, por exemplo: alunos, produtos, filmes, jogos, tarefas, cursos ou veículos.

Sua API deverá possuir, no mínimo:

- Um endpoint GET para listar os recursos.
- Um endpoint GET para consultar um recurso pelo id.
- Um endpoint POST para criar um recurso.
- Um endpoint PUT para atualizar um recurso.
- Um endpoint DELETE para excluir um recurso.
- Respostas em JSON.
- Tratamento de recurso não encontrado com código 404.

Exemplo para uma API de alunos:

```
GET     /alunos
GET     /alunos/{id}
POST    /alunos
PUT     /alunos/{id}
DELETE  /alunos/{id}
```

Exemplo de objeto JSON:

```
{
    "id": 1,
    "nome": "Maria",
    "curso": "Informática"
}
```

22. Desafio opcional

Para quem terminar antes, implemente uma das melhorias abaixo:

- Adicionar o campo ano ao livro.
- Criar uma busca por autor.
- Criar uma busca por parte do título.
- Adicionar validações adicionais aos dados recebidos.
- Pesquisar como implementar PATCH para atualização parcial.
- Pesquisar como conectar FastAPI a um banco de dados SQLite.

23. Checklist de entrega

☐ O servidor FastAPI inicia sem erros.

☐ A rota /docs abre corretamente.

☐ GET /livros funciona.

☐ GET /livros/{id} funciona.

☐ POST /livros cadastra um recurso.

☐ PUT /livros/{id} atualiza um recurso.

☐ DELETE /livros/{id} exclui um recurso.

☐ Recursos inexistentes retornam erro 404.

☐ O código está salvo e organizado.

24. Conclusão

Nesta prática, construímos uma API REST completa em Python utilizando FastAPI. A atividade permitiu aplicar, em código, os conceitos de HTTP, cliente e servidor, requisição e resposta, endpoints, JSON, métodos HTTP, CRUD e códigos de status. A partir dessa base, o próximo passo natural é estudar persistência de dados com banco de dados, autenticação, organização do projeto em módulos e implantação da API em um servidor.